

5.3 Function Philosophy

What makes a good function? The most important aspect of a good "building block" is that have a single, well-defined task to perform. When you find that a program is hard to manage, it's often because it has not been designed and broken up into functions cleanly. Two obvious reasons for moving code down into a function are because:

1. It appeared in the main program several times, such that by making it a function, it can be written just once, and the several places where it used to appear can be replaced with calls to the new function.
2. The main program was getting too big, so it could be made (presumably) smaller and more manageable by lopping part of it off and making it a function.

These two reasons are important, and they represent significant benefits of well-chosen functions, but they are not sufficient to automatically identify a good function. As we've been suggesting, a good function has at least these two additional attributes:

3. It does just one well-defined task, and does it well.
4. Its interface to the rest of the program is clean and narrow.

Attribute 3 is just a restatement of two things we said above. Attribute 4 says that you shouldn't have to keep track of too many things when calling a function. If you know what a function is supposed to do, and if its task is simple and well-defined, there should be just a few pieces of information you have to give it to act upon, and one or just a few pieces of information which it returns to you when it's done. If you find yourself having to pass lots and lots of information to a function, or remember details of its internal implementation to make sure that it will work properly this time, it's often a sign that the function is not sufficiently well-defined. (A poorly-defined function may be an arbitrary chunk of code that was ripped out of a main program that was getting too big, such that it essentially has to have access to all of that main function's local variables.)

The whole point of breaking a program up into functions is so that you don't have to think about the entire program at once; ideally, you can think about just one function at a time. We say that a good function is a "black box," which is supposed to suggest that the "container" it's in is opaque--callers can't see inside it (and the function inside can't see out). When you call a function, you only have to know what it does, not how it does it. When you're *writing* a function, you only have to know what it's supposed to do, and you don't have to know why or under what circumstances its caller will be calling it. (When designing a function, we should perhaps think about the callers just enough to ensure that the function we're designing will be easy to call, and that we aren't accidentally setting things up so that callers will have to think about any internal details.)

Some functions may be hard to write (if they have a hard job to do, or if it's hard to make them do it truly well), but that difficulty should be compartmentalized along with the function itself. Once you've written a "hard" function, you should be able to sit back and relax and watch it do that hard work on call from the rest of your program. It should be pleasant to notice (in the ideal case) how much easier the rest of the program is to write, now that the hard work can be deferred to this workhorse function.

(In fact, if a difficult-to-write function's interface is well-defined, you may be able to get away with writing a quick-and-dirty version of the function first, so that you can begin testing the rest of the program, and then go back later and rewrite the function to do the hard parts. As long as the function's original interface anticipated the hard parts, you won't have to rewrite the rest of the

program when you fix the function.)

What I've been trying to say in the preceding few paragraphs is that functions are important for far more important reasons than just saving typing. Sometimes, we'll write a function which we only call once, just because breaking it out into a function makes things clearer and easier.

If you find that difficulties pervade a program, that the hard parts can't be buried inside black-box functions and then forgotten about; if you find that there are hard parts which involve complicated interactions among multiple functions, then the program probably needs redesigning.

For the purposes of explanation, we've been seeming to talk so far only about ``main programs" and the functions they call and the rationale behind moving some piece of code down out of a ``main program" into a function. But in reality, there's obviously no need to restrict ourselves to a two-tier scheme. Any function we find ourself writing will often be appropriately written in terms of sub-functions, sub-sub-functions, etc. (Furthermore, the ``main program," `main()`, is itself just a function.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)